

Python E-Course

Module 5 — Geometric Transformations

Detailed Notes

Module Overview

Level: Color & Geometry

Duration: ~1 week

Prerequisites: Modules 1-4

What You'll Learn

- Translate, rotate, and scale images correctly.
- Build and apply 2×3 affine transforms.
- Use perspective (4-point) transforms for document scanning.
- Compare interpolation methods (NEAREST, LINEAR, CUBIC, LANCZOS).
- Resize, crop, and flip.

Lessons

#	Lesson	Duration
1	Translation, Rotation, Scaling	30 min
2	The 2×3 Affine Matrix	30 min
3	Perspective Transformation	35 min
4	Resizing and Interpolation	30 min
5	Flip, Crop, Pad	20 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 6: Image Enhancement

Lesson 1: Translation, Rotation, Scaling

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Build translation matrices.
- Rotate images with `cv2.getRotationMatrix2D` + `cv2.warpAffine`.
- Scale via `cv2.resize` or the affine matrix.
- Anticipate clipping when transforms push pixels off the canvas.

Prerequisites

- Module 1 coordinate systems

1. Why This Matters

Translating / rotating / scaling are the most common geometric ops — for data augmentation, alignment, and display. The math (a 2×3 matrix) is simple; the gotchas (canvas size, center of rotation) cause most bugs.

2. Concept

2.1 Translation = adding (tx, ty)

Matrix form:

$$M = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \end{bmatrix}$$

In code:

```
M = np.float32([[1, 0, tx], [0, 1, ty]])
out = cv2.warpAffine(img, M, (w, h))
```

(w, h) is the output size in pixels.

2.2 Rotation around a point

```
M = cv2.getRotationMatrix2D(center=(cx, cy), angle=30, scale=1.0)
out = cv2.warpAffine(img, M, (w, h))
```

- Angle in degrees, counter-clockwise.
- scale lets you zoom in/out simultaneously.

Default rotation around the centre:

```
h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)
```

2.3 Canvas size after rotation

Rotated content often falls outside the original canvas. To compute the bounding box that fits the whole rotated image:

```
import numpy as np
def rotate_keep(img, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    new_w = int(h * sin + w * cos)
    new_h = int(h * cos + w * sin)
    M[0, 2] += (new_w / 2) - w / 2      # shift to centre in new canvas
    M[1, 2] += (new_h / 2) - h / 2
    return cv2.warpAffine(img, M, (new_w, new_h))
```

2.4 Scaling via cv2.resize

```
out = cv2.resize(img, (new_w, new_h))
out = cv2.resize(img, None, fx=0.5, fy=0.5)
```

For pure scaling, cv2.resize is simpler and faster than warpAffine.

2.5 borderMode — what's outside the image

cv2.warpAffine(..., borderMode=cv2.BORDER_CONSTANT, borderValue=(0, 0, 0))

Default is black. Other useful options:

- BORDER_REPLICATE — copy the edge pixel outward
- BORDER_REFLECT — mirror
- BORDER_WRAP — tile

3. Code Examples

Example 1: Translate by (50, 30)

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
M = np.float32([[1, 0, 50], [0, 1, 30]])
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("translated.png", out)
```

Expected Output: Astronaut shifted 50 px right and 30 px down; the freshly-uncovered area on the top-left is black (border default).

Example 2: Rotate 45° around centre

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
```

```

h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1.0)
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("rotated45.png", out)

```

Expected Output: Astronaut rotated 45° counter-clockwise around the centre, with the four corners pushed off the canvas (black triangles at the corners).

Example 3: Rotate without clipping

```

import cv2, numpy as np
from skimage.data import astronaut

def rotate_keep(img, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w/2, h/2), angle, 1.0)
    cos, sin = abs(M[0, 0]), abs(M[0, 1])
    new_w = int(h * sin + w * cos)
    new_h = int(h * cos + w * sin)
    M[0, 2] += (new_w / 2) - w / 2
    M[1, 2] += (new_h / 2) - h / 2
    return cv2.warpAffine(img, M, (new_w, new_h))

out = rotate_keep(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR), 30)
print("output shape:", out.shape)
cv2.imwrite("rotated30_keep.png", out)

```

Expected Output: A larger canvas with the rotated astronaut fully visible inside. output shape: will be (700-ish, 700-ish, 3) instead of the original 512².

Example 4: Scale by 0.5

```

import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
small = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
print(img.shape, "->", small.shape)
cv2.imwrite("coffee_half.png", small)

```

Expected Output:

(400, 600, 3) -> (200, 300, 3)

4. Parameter Sensitivity

Param	Effect
Angle	how much rotation (CCW positive)
Scale (in getRotationMatrix2D)	uniform zoom
Center	pivot point
borderMode	what's outside the source image

5. Common Mistakes

Mistake	What Happens	Fix
Wrong dsize argument order	Stretches one axis	(W, H) not (H, W)
Rotating then losing corners	Image is clipped	Compute new canvas size
Angle in radians	Tiny / weird rotation	OpenCV uses degrees
Translating in wrong direction	Confusion	tx positive = right; ty positive = down

6. Practice Tasks

- Translate astronaut() by 100 px right, 0 down.
- Rotate coffee() 15° around its centre, keep original canvas size.
- Rotate coffee() 30° without clipping — write a rotate_keep helper.

7. Key Takeaways

- Translation matrix: `[[1,0,tx],[0,1,ty]]`.
- Rotation matrix: `cv2.getRotationMatrix2D(center, angle_deg, scale)`.
- `cv2.warpAffine` needs an output dsize=(W, H).
- Rotating without resizing clips; build a new canvas if you need everything visible.

8. What's Next

The 2×3 affine matrix in depth — combining translate / rotate / shear / scale into one.

Lesson 2: The 2×3 Affine Matrix

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Read the meaning of every number in a 2×3 affine matrix.
- Build affine matrices from 3 source/destination point pairs.
- Combine multiple affine transforms.

Prerequisites

- Module 5 Lesson 1, reference/math-refresher.md §5

1. Why This Matters

An affine transform packs translate + rotate + scale + shear into one matrix. Knowing what each number does lets you debug "why is my warp diagonal", build custom transforms, and compose ops without applying them one at a time.

2. Concept

2.1 The matrix

$$M = \begin{bmatrix} a & b & tx \\ c & d & ty \end{bmatrix}$$

- a, d — scale on x and y respectively (1.0 = no scale).
- b, c — shear.
- tx, ty — translation in pixels.

Mathematically: $[x'; y'; 1] = M \cdot [x; y; 1]$ (with an implicit $[0 \ 0 \ 1]$ row).

2.2 Pure forms

Op	Matrix
Identity	[[1,0,0],[0,1,0]]
Translate (tx, ty)	[[1,0,tx],[0,1,ty]]
Scale (sx, sy)	[[sx,0,0],[0,sy,0]]
Rotate θ (CCW)	[[cos θ , -sin θ , 0], [sin θ , cos θ , 0]]
Shear x by k	[[1,k,0],[0,1,0]]
Shear y by k	[[1,0,0],[k,1,0]]
Reflect over y-axis (flip horizontal)	[[-1,0,W], [0,1,0]]

(Reflection requires translating after flipping to keep content visible.)

2.3 From 3 point pairs — cv2.getAffineTransform

If you know where 3 source points should land in the destination, OpenCV finds the unique affine matrix that does it:

```
src = np.float32([[0, 0], [w, 0], [0, h]])          # top-left, top-right, bottom-
left
dst = np.float32([[0, h*0.3], [w*0.8, 0], [w*0.2, h]])
M = cv2.getAffineTransform(src, dst)
out = cv2.warpAffine(img, M, (w, h))
```

Affine has 6 unknowns (the 6 numbers of M); 3 point pairs $\times$ 2 coords = 6 equations. So 3 pairs uniquely define an affine.

For 4+ pairs use cv2.estimateAffinePartial2D / cv2.estimateAffine2D (least-squares fit).

2.4 Composing transforms

Combine ops by multiplying matrices. OpenCV doesn't have a built-in multiply for the 2 $\times$ 3 form (it's a chopped 3 $\times$ 3); the cleanest way is to pad to 3 $\times$ 3, multiply with NumPy, and slice back:

```
def compose(*Ms):
    result = np.eye(3, dtype=np.float32)
    for M in Ms:
        M3 = np.vstack([M, [0, 0, 1]])
        result = M3 @ result
    return result[:2]
```

Then cv2.warpAffine(img, compose(M2, M1), dsize) applies M1 first then M2.

2.5 The order matters

Matrix multiplication is non-commutative. rotate(translate(img)) $\neq$ translate(rotate(img)).

3. Code Examples

Example 1: Custom affine via 3 points

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

src = np.float32([[0, 0], [w, 0], [0, h]])
dst = np.float32([[40, 30], [w - 10, 60], [60, h - 20]])
M = cv2.getAffineTransform(src, dst)
print("M =\n", M)

out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("affine_custom.png", out)
```

Expected Output: Astronaut warped — top-left corner pulled inward, top-right pulled down, bottom-left pulled right. Console prints a 2 $\times$ 3 matrix with non-zero tx, ty.

Example 2: Manual shear

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

k = 0.3
M = np.float32([[1, k, 0], [0, 1, 0]])
out = cv2.warpAffine(img, M, (w + int(h * k), h))
cv2.imwrite("sheared.png", out)
```

Expected Output: Image sheared horizontally — top stays put, bottom shifted right (or vice versa depending on k sign). Canvas widened so nothing is clipped.

Example 3: Compose translate + rotate

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

# 1. translate so the "centre of interest" is at the origin
T1 = np.float32([[1, 0, -w/2], [0, 1, -h/2]])
# 2. rotate by angle around the new origin
ang = np.deg2rad(20)
R = np.float32([[np.cos(ang), -np.sin(ang), 0],
                [np.sin(ang), np.cos(ang), 0]])
# 3. translate back
T2 = np.float32([[1, 0, w/2], [0, 1, h/2]])

def compose(*Ms):
    result = np.eye(3, dtype=np.float32)
    for M in Ms:
        result = np.vstack([M, [0, 0, 1]]) @ result
    return result[:2]

M = compose(T1, R, T2)
out = cv2.warpAffine(img, M, (w, h))
cv2.imwrite("composed.png", out)
```

Expected Output: Same as `cv2.getRotationMatrix2D(center=(w/2, h/2), angle=20, scale=1)` would have done — demonstrates that compose gives you full control.

4. Parameter Sensitivity

Matrix element	Larger value	Smaller value
a (sx)	wider	narrower / squashed
d (sy)	taller	shorter
b (shear x)	more lean to the right	leans left if negative
c (shear y)	leans down to the right	inverse

tx, ty	shift further	less shift
--------	---------------	------------

5. Common Mistakes

Mistake	What Happens	Fix
Pass dst, src in wrong order to getAffineTransform	Inverse transform	Match the call signature
Forget to pad to 3×3 before matrix-multiplying	Wrong composition	Use a helper like compose
Apply rotation without recentring	Image swings off-canvas	Translate to origin, rotate, translate back

6. Practice Tasks

- Build a translate-only affine $[[1,0,40],[0,1,-20]]$. Apply it.
- Use `cv2.getAffineTransform` with three custom point pairs of your choice.
- Compose a 30° rotation around the centre using the compose helper.

7. Key Takeaways

- 2×3 affine: 6 numbers (scale x/y, shear x/y, translate x/y).
- `cv2.getAffineTransform(src, dst)` — solve from 3 point pairs.
- Compose by multiplying 3×3 forms.
- Order matters: translate-rotate ≠ rotate-translate.

8. What's Next

Perspective — when 4 points are needed (affine is too restrictive for tilt).

Lesson 3: Perspective Transformation

Module: 5 — Geometric Transformations

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Distinguish affine from perspective.
- Build a perspective transform from 4 point pairs.
- Implement a document scanner (4-point straighten).

Prerequisites

- Module 5 Lesson 2

1. Why This Matters

Affine keeps parallel lines parallel. Perspective doesn't — which is what you need when fixing a phone photo of a tilted document: the page's parallel edges converge in the image, and you need to map them back to a rectangle.

2. Concept

2.1 The 3×3 perspective matrix

Where affine is 2×3, perspective is 3×3 with 8 free parameters (the 9th is normalised).

Mathematically:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Then $x_{\text{out}} = x'/w'$, $y_{\text{out}} = y'/w'$. The non-zero g, h are what give perspective its "tilt".

2.2 4 point pairs uniquely define it

```
src = np.float32([[x1,y1], [x2,y2], [x3,y3], [x4,y4]])
dst = np.float32([[X1,Y1], [X2,Y2], [X3,Y3], [X4,Y4]])
M = cv2.getPerspectiveTransform(src, dst)
out = cv2.warpPerspective(img, M, (W, H))
```

(W, H) is the output canvas — typically the size of the rectangle you're mapping to.

2.3 Document-scanner recipe

Goal: photo of a tilted document → flat rectangular scan.

- Detect the 4 corners of the document (manually or with edge detection, M9).
- Order them consistently: top-left, top-right, bottom-right, bottom-left.
- Decide output size — usually (max_width, max_height) from corner distances.

- cv2.getPerspectiveTransform + cv2.warpPerspective.

2.4 Corner ordering helper

A simple recipe: top-left has the smallest x+y, bottom-right the largest. Top-right has the smallest y-x, bottom-left the largest.

```
def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
        pts[np.argmin(s)],      # top-left
        pts[np.argmax(d)],      # top-right
        pts[np.argmax(s)],      # bottom-right
        pts[np.argmin(d)],      # bottom-left
    ], dtype=np.float32)
```

2.5 Output size from corners

```
(tl, tr, br, bl) = ordered
width  = int(max(np.linalg.norm(tr - tl), np.linalg.norm(br - bl)))
height = int(max(np.linalg.norm(bl - tl), np.linalg.norm(br - tr)))
```

This preserves the document's actual proportions.

3. Code Examples

Example 1: Bird's-eye warp of an arbitrary 4-point region

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]

# Pick 4 corners of a "trapezoid" in the image
src = np.float32([[100, 100], [400, 80], [450, 380], [50, 400]])
# Map to a flat 300x300 rectangle
dst = np.float32([[0, 0], [300, 0], [300, 300], [0, 300]])

M = cv2.getPerspectiveTransform(src, dst)
out = cv2.warpPerspective(img, M, (300, 300))
cv2.imwrite("warped.png", out)
```

Expected Output: A 300×300 image — the trapezoid region of the astronaut "straightened" into a square.

Example 2: Order corners helper

```
import numpy as np

def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
```

```

        pts[np.argmax(s)],
        pts[np.argmax(d)],
        pts[np.argmax(s)],
        pts[np.argmax(d)],
    ], dtype=np.float32)

raw = np.array([[400, 80], [50, 400], [100, 100], [450, 380]])
print(order_corners(raw))

```

Expected Output:

```

[[100. 100.]
 [400.  80.]
 [450. 380.]
 [ 50. 400.]]

```

Same four points, in TL, TR, BR, BL order.

Example 3: Synthetic "document" scanner

```

import cv2, numpy as np

# Build a fake skewed "document" on a background
canvas = np.full((500, 700, 3), 50, dtype=np.uint8)
src_pts = np.float32([[120, 80], [620, 120], [580, 460], [100, 420]])
doc_pts = np.float32([[0, 0], [400, 0], [400, 500], [0, 500]])

# Create a small "page" image to project into the canvas
page = np.full((500, 400, 3), 240, dtype=np.uint8)
cv2.putText(page, "DOCUMENT", (50, 250),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (30, 30, 30), 4)

# Project page onto canvas
M_into = cv2.getPerspectiveTransform(doc_pts, src_pts)
warped_page = cv2.warpPerspective(page, M_into, (canvas.shape[1], canvas.shape[0]))
mask = (warped_page.sum(axis=2) > 0).astype(np.uint8) * 255
canvas[mask > 0] = warped_page[mask > 0]
cv2.imwrite("skewed_doc.png", canvas)

# Now "scan": reverse the transform from src_pts -> flat doc_pts
M_back = cv2.getPerspectiveTransform(src_pts, doc_pts)
scanned = cv2.warpPerspective(canvas, M_back, (400, 500))
cv2.imwrite("scanned.png", scanned)

```

Expected Output:

- skewed_doc.png: dark canvas with a skewed white page reading "DOCUMENT".
- scanned.png: 400×500 image of the page straightened out, "DOCUMENT" perfectly readable.

4. Parameter Sensitivity

Param	Effect
dst size (W, H)	Output resolution

corner ordering	Wrong order = mirror / flip
extreme tilt	Bad output — perspective loses precision when source is near-degenerate

5. Common Mistakes

Mistake	What Happens	Fix
Wrong corner order	Output is flipped or twisted	Use <code>order_corners</code>
dst size doesn't match aspect	Stretched output	Compute size from corner distances
Using 3 points (affine sig) for perspective	Wrong function	<code>cv2.getPerspectiveTransform</code> needs 4
Passing int arrays	"wrong type" error	Cast to <code>np.float32</code>

6. Practice Tasks

- Pick 4 points on any image, warp to a 200×200 square.
- Implement `order_corners`; verify on a shuffled 4-point set.
- Build the document-scanner round-trip: skew a page into a canvas, then un-skew.

7. Key Takeaways

- Perspective uses a 3×3 matrix; 4 point pairs define it.
- `cv2.getPerspectiveTransform(src, dst) + cv2.warpPerspective`.
- Document scanning = 4-corner perspective straighten.
- Order corners consistently for predictable output.

8. What's Next

Resizing & interpolation — when shrinking and enlarging, which interpolation should you use?

Lesson 4: Resizing and Interpolation

Module: 5 — Geometric Transformations

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use `cv2.resize` correctly (argument order matters).
- Choose the right interpolation: `NEAREST`, `LINEAR`, `CUBIC`, `LANCZOS4`, `AREA`.
- Pick `INTER_AREA` for downsizing, `INTER_CUBIC` / `LANCZOS` for upsizing.

Prerequisites

- Module 5 Lesson 3

1. Why This Matters

Resize is the most-called geometric op — to fit a model input, generate thumbnails, save memory. The default interpolation isn't always right; the wrong choice gives mushy or jaggy output.

2. Concept

2.1 Two signature forms

```
cv2.resize(img, dsize, interpolation=cv2.INTER_LINEAR)
cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
```

`dsize` is (W, H) — opposite of `.shape`. `fx`, `fy` are multiplicative scale factors.

2.2 The 5 interpolation modes

Mode	Cost	Quality going UP	Quality going DOWN
<code>INTER_NEAREST</code>	cheapest	blocky	aliasing
<code>INTER_LINEAR</code> (default)	cheap	smooth, slightly blurry	decent
<code>INTER_CUBIC</code>	medium	sharper than linear	decent
<code>INTER_LANCZOS4</code>	expensive	sharpest	excellent but slow
<code>INTER_AREA</code>	medium	terrible (don't)	best for downsizing

2.3 The shortcut

- Going down (smaller) → `INTER_AREA`.
- Going up (larger) → `INTER_CUBIC` (good default) or `INTER_LANCZOS4` (slow but sharpest).
- Pixel art / masks where blocky is intentional → `INTER_NEAREST`.

2.4 What's actually happening

- `NEAREST` picks the closest source pixel.
- `LINEAR` averages 2×2 source neighbours.

- CUBIC fits a smoother polynomial over 4×4 neighbours.
- LANCZOS4 uses a sinc kernel over 8×8 neighbours.
- AREA averages all source pixels that fall into each destination pixel — proper for downscaling because it's an actual low-pass filter (no aliasing).

2.5 Anti-aliasing on downscale

If you use LINEAR or CUBIC to downscale by a lot, you get aliasing (moiré patterns on fine textures). AREA prevents this.

3. Code Examples

Example 1: Same downscale, different methods

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
for name, mode in [("NEAREST", cv2.INTER_NEAREST),
                  ("LINEAR", cv2.INTER_LINEAR),
                  ("CUBIC", cv2.INTER_CUBIC),
                  ("AREA", cv2.INTER_AREA),
                  ("LANCZOS", cv2.INTER_LANCZOS4)]:
    small = cv2.resize(img, (64, 64), interpolation=mode)
    cv2.imwrite(f"down_{name}.png", small)
    print(name, small.shape)
```

Expected Output: Five 64×64 thumbnails. NEAREST is blocky and aliased; AREA looks smoothest and faithful; the others are similar to each other (LINEAR slightly softer than CUBIC / LANCZOS).

Example 2: Same upscale

```
import cv2
from skimage.data import coins

img = coins()
small = cv2.resize(img, (64, 64), interpolation=cv2.INTER_AREA)

for name, mode in [("NEAREST", cv2.INTER_NEAREST),
                  ("LINEAR", cv2.INTER_LINEAR),
                  ("CUBIC", cv2.INTER_CUBIC),
                  ("LANCZOS", cv2.INTER_LANCZOS4)]:
    big = cv2.resize(small, (256, 256), interpolation=mode)
    cv2.imwrite(f"up_{name}.png", big)
```

Expected Output: Four 256×256 upscales. NEAREST looks pixelated (each source pixel is a 4×4 block). LINEAR is smooth but soft. CUBIC sharper. LANCZOS sharpest but with the slowest run time.

Example 3: Don't use AREA for upscaling

```
import cv2
from skimage.data import camera
```



```

img = camera()
small = cv2.resize(img, (50, 50), interpolation=cv2.INTER_AREA)

big_area = cv2.resize(small, (400, 400), interpolation=cv2.INTER_AREA)
big_cubic = cv2.resize(small, (400, 400), interpolation=cv2.INTER_CUBIC)
cv2.imwrite("up_area.png", big_area)
cv2.imwrite("up_cubic.png", big_cubic)

```

Expected Output: AREA upscale looks blocky / averaged poorly; CUBIC looks smooth.

Example 4: Maintain aspect ratio

```

import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
h, w = img.shape[:2]
target_w = 400
ratio = target_w / w
out = cv2.resize(img, (target_w, int(h * ratio)), interpolation=cv2.INTER_AREA)
print(img.shape, "->", out.shape)

```

Expected Output: (400, 600, 3) -> (266, 400, 3) (aspect preserved).

4. Parameter Sensitivity

Mode	Best at	Worst at
NEAREST	Pixel art, mask scaling	Photo upscale
LINEAR	Generic default	Sharp results when upscaling
CUBIC	Photo upscale	Slow for video
LANCZOS4	Highest quality upscale	Slowest
AREA	Photo downscale	Any upscale

5. Common Mistakes

Mistake	What Happens	Fix
(H, W) in dsize	Stretched	(W, H)
LINEAR for huge downscale	Aliasing	AREA
AREA for upscale	Blocky	CUBIC / LANCZOS
Forgetting aspect ratio	Squashed image	Compute one axis from the other

6. Practice Tasks

- Resize astronaut() to 100×100 with each of the 5 modes; compare.
- Upscale that 100×100 back to 512×512 with NEAREST and LANCZOS; compare.
- Resize coffee() so the width is 300, preserving aspect ratio.

7. Key Takeaways

- cv2.resize(..., (W, H), interpolation=...).
- Downscale: INTER_AREA. Upscale: INTER_CUBIC or INTER_LANCZOS4.

- Pixel art / masks: INTER_NEAREST.
- Maintain aspect ratio manually.

8. What's Next

Flip, crop, and pad — the small but ever-present resize cousins.

Lesson 5: Flip, Crop, Pad

Module: 5 — Geometric Transformations

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Flip images horizontally / vertically / both with `cv2.flip`.
- Crop with NumPy slicing.
- Pad with `cv2.copyMakeBorder` and the various border modes.

Prerequisites

- Module 5 Lesson 4

1. Why This Matters

Three small ops you'll use constantly — for data augmentation, batch alignment, and resolving size mismatches before stacking / concatenating.

2. Concept

2.1 Flip

```
cv2.flip(img, 0)    # vertical flip (around x-axis)
cv2.flip(img, 1)    # horizontal flip (around y-axis)
cv2.flip(img, -1)   # both
```

Mnemonic: positive = horizontal mirror (most useful for augmentation).

2.2 Crop = slicing

```
crop = img[y1:y2, x1:x2]
```

Already covered (M3 L3). Worth repeating in the geometry context.

2.3 Pad — `cv2.copyMakeBorder`

```
out = cv2.copyMakeBorder(img, top, bottom, left, right,
                          borderType, value=0)
```

`borderType` options:

Type	Effect
<code>cv2.BORDER_CONSTANT</code>	Solid color (use <code>value=...</code>)
<code>cv2.BORDER_REPLICATE</code>	Repeat the edge pixel outward
<code>cv2.BORDER_REFLECT</code>	Mirror
<code>cv2.BORDER_REFLECT_101</code>	Mirror without repeating the edge (default for many filters)
<code>cv2.BORDER_WRAP</code>	Tile the image

2.4 Resize-with-padding (letterbox)

A common trick: fit an image into a target size by resizing the longest side and padding the rest:

```
def letterbox(img, target=(224, 224), color=(0, 0, 0)):  
    th, tw = target  
    h, w = img.shape[:2]  
    s = min(tw / w, th / h)  
    nw, nh = int(w * s), int(h * s)  
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_AREA)  
    top = (th - nh) // 2  
    bottom = th - nh - top  
    left = (tw - nw) // 2  
    right = tw - nw - left  
    return cv2.copyMakeBorder(resized, top, bottom, left, right,  
                              cv2.BORDER_CONSTANT, value=color)
```

This preserves aspect ratio — used a lot in ML pipelines that need a fixed input size.

3. Code Examples

Example 1: Horizontal flip (data augmentation)

```
import cv2  
from skimage.data import astronaut  
  
img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)  
mirrored = cv2.flip(img, 1)  
cv2.imwrite("astro_flip_h.png", mirrored)
```

Expected Output: Astronaut mirrored left-to-right.

Example 2: All four flips

```
import cv2, numpy as np  
from skimage.data import coffee  
  
img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)  
out = np.hstack([img, cv2.flip(img, 1), cv2.flip(img, 0), cv2.flip(img, -1)])  
cv2.imwrite("flips.png", out)
```

Expected Output: 4-up: original, horizontal mirror, vertical mirror, both axes.

Example 3: Pad with each border type

```
import cv2, numpy as np  
from skimage.data import camera  
  
img = camera()  
common = dict(top=30, bottom=30, left=30, right=30)  
panels = [  
    ("CONSTANT", cv2.copyMakeBorder(img, **common,  
borderType=cv2.BORDER_CONSTANT, value=128)),  
    ("REPLICATE", cv2.copyMakeBorder(img, **common,  
borderType=cv2.BORDER_REPLICATE)),  
    ("REFLECT", cv2.copyMakeBorder(img, **common,
```

```

borderType=cv2.BORDER_REFLECT)),
    ("REFLECT_101", cv2.copyMakeBorder(img, **common,
borderType=cv2.BORDER_REFLECT_101)),
    ("WRAP",          cv2.copyMakeBorder(img, **common, borderType=cv2.BORDER_WRAP)),
]
for name, p in panels:
    cv2.imwrite(f"pad_{name}.png", p)

```

Expected Output: Five files showing the different border behaviours — CONSTANT is gray, REPLICATE stretches the edge outward, REFLECT mirrors, WRAP tiles.

Example 4: Letterbox to square

```

import cv2
from skimage.data import coffee

def letterbox(img, target=(300, 300), color=(0, 0, 0)):
    th, tw = target
    h, w = img.shape[:2]
    s = min(tw / w, th / h)
    nw, nh = int(w * s), int(h * s)
    resized = cv2.resize(img, (nw, nh), interpolation=cv2.INTER_AREA)
    top     = (th - nh) // 2
    bottom  = th - nh - top
    left    = (tw - nw) // 2
    right   = tw - nw - left
    return cv2.copyMakeBorder(resized, top, bottom, left, right,
                              cv2.BORDER_CONSTANT, value=color)

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
out = letterbox(img, (300, 300))
print(out.shape)      # (300, 300, 3)
cv2.imwrite("letterbox.png", out)

```

Expected Output: A 300×300 image with the coffee photo centred and black bars filling the unused vertical space.

4. Parameter Sensitivity

(Each op is its own thing — no shared param table.)

5. Common Mistakes

Mistake	What Happens	Fix
cv2.flip(img, 0) thinking it's horizontal	It's vertical	Remember: flipCode == 1 for horizontal
Pad with negative border size	Crops instead	Use slicing for crop
Letterbox stretching	Aspect ratio lost	Pick s = min(...), not max
BORDER_CONSTANT without value	Black border	Pass value=(B,G,R) for color

6. Practice Tasks

- Flip astronaut() along all 3 axes (h, v, both) and save each.
- Pad coins() with 30 px black borders on all sides.
- Letterbox coffee() into a 256×256 square with white padding.

7. Key Takeaways

- cv2.flip(img, code): 0 vertical, 1 horizontal, -1 both.
- Crop = slicing.
- cv2.copyMakeBorder for padding; many border types.
- Letterbox preserves aspect ratio when fitting to a target size.

8. What's Next

End of Module 5. Next: Module 6 — Image Enhancement — histograms, equalisation, CLAHE.

Module 5 — Exercises

15 exercises (3 per lesson).

5.1 Translate/Rotate/Scale

- (E) Translate astronaut() by (100, 0).
- (M) Rotate coffee() 30° around its centre, keep original canvas.
- (H) Rotate coffee() 30° without clipping (rotate_keep).

5.2 Affine

- (E) Build a translate-only affine $\begin{bmatrix} 1 & 0 & 40 \\ 0 & 1 & -20 \end{bmatrix}$. Apply.
- (M) Use cv2.getAffineTransform with 3 custom point pairs.
- (H) Compose a 30° rotation about the centre with the compose helper.

5.3 Perspective

- (E) Warp a 4-point region into a 200×200 square.
- (M) Implement order_corners; test on a shuffled list.
- (H) Build the document-scanner round-trip (skew then un-skew).

5.4 Resize

- (E) Resize astronaut() to 100×100 with each of NEAREST/LINEAR/CUBIC/AREA/LANCZOS4. Compare.
- (M) Upscale a small image to 512² with NEAREST vs LANCZOS. Compare.
- (H) Resize coffee() so width is 300 and aspect ratio is preserved.

5.5 Flip / Crop / Pad

- (E) Flip astronaut() along all 3 axes.
- (M) Pad coins() with 30 px black borders.
- (H) Letterbox coffee() to a 256×256 square with white padding.

Module 5 — Quiz

10 MCQs.

Q1

cv2.warpAffine expects dsize as: A. (H, W) B. (W, H) C. depends D. (W, H, C)

Answer: B (L1)

Q2

cv2.getRotationMatrix2D(center, angle, scale) angle is in: A. radians B. degrees C. percent D. cm

Answer: B (L1)

Q3

Affine has how many free parameters? A. 4 B. 6 C. 8 D. 9

Answer: B (L2)

Q4

Perspective has how many free parameters? A. 4 B. 6 C. 8 D. 12

Answer: C (L3)

Q5

For downscaling photos, the recommended interpolation is: A. NEAREST B. LINEAR C. AREA D. CUBIC

Answer: C (L4)

Q6

For sharp upscaling, you'd use: A. NEAREST B. LINEAR C. AREA D. LANCZOS4

Answer: D (L4)

Q7

cv2.flip(img, 1) does: A. vertical flip B. horizontal flip C. both axes D. no-op

Answer: B (L5)

Q8

For a document scanner (tilted photo → flat rectangle), use: A. rotation B. affine C. perspective D. resize

Answer: C (L3)

Q9

cv2.getAffineTransform needs how many point pairs? A. 2 B. 3 C. 4 D. 6

Answer: B (L2)

Q10

cv2.copyMakeBorder(..., BORDER_REFLECT) produces: A. solid color B. mirrored edge C. repeated edge pixel D. no border

Answer: B (L5)

Module 5 — Assignment: Image Aligner

Estimated time: 2 hours

Combines: Module 5 + Modules 1-4

Brief

Build a CLI that aligns an input image to a reference by user-supplied corresponding points.

Requirements

- align.py INPUT REFERENCE --points "x1,y1;x2,y2;x3,y3" (3 pairs in input AND 3 in reference, or 4 if --perspective).
- Use cv2.getAffineTransform (default) or cv2.getPerspectiveTransform (--perspective).
- Warp INPUT to align with REFERENCE.
- Save side-by-side comparison: original | aligned | reference.

Constraints

- Module 5 only.
- Use argparse to parse point strings.
- Validate 3 (affine) or 4 (perspective) point pairs.

Expected Behaviour

```
$ python align.py input.png reference.png \  
  --input-points "0,0;500,0;0,500" \  
  --ref-points   "20,40;510,10;30,495"
```

```
saved -> comparison.png  
warp type: affine
```

Grading Rubric

Criterion	Weight
Affine path works	25%
Perspective path works	25%
Point parsing + validation	20%
Side-by-side output	15%
Style + argparse	15%

Submission

Save as assignment_module5.py.

Module 5 — Mini Project: 4-Point Document Scanner

Overview

Build a tool that takes a photo of a tilted document and produces a flat, rectangular "scan". The 4 corners are supplied by the user (click in a window).

Concepts Used

- Perspective transform (M5 L3)
- Mouse callbacks (M2 L3)
- Corner ordering

Features

- scanner.py INPUT_IMAGE
- Opens the image; user clicks 4 corners.
- Auto-orders corners (TL, TR, BR, BL).
- Computes output size from the corner distances.
- Warps to flat rectangle.
- Saves scanned.png.

Starter Code

scanner.py:

```
import sys
import cv2
import numpy as np

def order_corners(pts):
    s = pts.sum(axis=1)
    d = np.diff(pts, axis=1).ravel()
    return np.array([
        pts[np.argmin(s)], pts[np.argmax(d)],
        pts[np.argmax(s)], pts[np.argmin(d)],
    ], dtype=np.float32)

def scan(img, pts):
    ordered = order_corners(np.array(pts))
    tl, tr, br, bl = ordered
    w = int(max(np.linalg.norm(tr - tl), np.linalg.norm(br - bl)))
    h = int(max(np.linalg.norm(bl - tl), np.linalg.norm(br - tr)))
    dst = np.float32([[0, 0], [w, 0], [w, h], [0, h]])
    M = cv2.getPerspectiveTransform(ordered, dst)
    return cv2.warpPerspective(img, M, (w, h))

def run(path):
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(path)
    if max(img.shape) > 1200:
```

```

img = cv2.resize(img, None, fx=0.5, fy=0.5)

pts = []
disp = img.copy()

def on_mouse(event, x, y, flags, param):
    nonlocal disp
    if event == cv2.EVENT_LBUTTONDOWN and len(pts) < 4:
        pts.append([x, y])
        cv2.circle(disp, (x, y), 5, (0, 255, 0), -1)
        if len(pts) > 1:
            cv2.line(disp, tuple(pts[-2]), tuple(pts[-1]), (0, 255, 0), 1)
        if len(pts) == 4:
            cv2.line(disp, tuple(pts[3]), tuple(pts[0]), (0, 255, 0), 1)

cv2.namedWindow("scan")
cv2.setMouseCallback("scan", on_mouse)
while True:
    cv2.imshow("scan", disp)
    if cv2.waitKey(20) & 0xFF == ord('q'):
        break
    if len(pts) == 4:
        cv2.waitKey(500)
        break
cv2.destroyAllWindows()

if len(pts) == 4:
    scanned = scan(img, pts)
    cv2.imwrite("scanned.png", scanned)
    print("saved -> scanned.png shape:", scanned.shape)
else:
    print("need 4 points; quit early")

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python scanner.py phone_photo.jpg
(window opens; click 4 corners of document)
saved -> scanned.png shape: (800, 600, 3)

```

Extension Challenges

- Auto-detect 4 corners with edge detection + contour analysis (preview of M9-M12).
- Add adaptive thresholding to the result for "black ink on white paper" look (preview of M11).
- Support saving multi-page PDFs.

Grading Rubric

Criterion	Weight
4-point click works	25%
Correct warp	30%

Corner ordering	15%
Output size from corners	15%
Code clarity	15%